



CS100 Recitation 2

Data Types, Literals & Operators

@lichf2025@shanghaitech.edu.cn

@heyx2025@shanghaitech.edu.cn

Contents

Contents

- 1 CS100 教学理念 ->
- 2 Lecture Notes ->: Undefined Behavior, 整型提升, 二进制数表示
- 3 Arithmetic Types ->: `sizeof`, 浮点数误差
- 4 Literals ->: 整型, 浮点型, 字符面值
- 5 Operators ->: 优先级, 结合性, 求值顺序, 位运算符

1. CS100 教学理念

1. CS100 教学理念

☒ Understand & Practice

纸上得来终觉浅，绝知此事要躬行。

- 上节课有同学反映：上课听的时候感觉接受比较困难，得自己下去重新研读一遍 Slides。
 - 个人观点：每个同学的接受知识的节奏不同，大家上课只需要把握到主要内容即可。
 - 不必强求自己在课上就完全理解所有内容！
 - 需要重新研读 Slides 是正常现象，大家不必焦虑
- 从这节课开始，我们会在习题课中增加若干习题。我会提供习题的讲解，大家可以听一遍，然后回去自行回顾一下！



- Feel Free~

2. 课程讲义 Review

2. 课程讲义 Review



Overview of Lecture Notes

1. Undefined behavior (未定义行为) ->
2. *Integer promotion (整型提升) ->
3. *Binary number representation (二进制数表示): sign-and-magnitude (原码)、one's complement (反码)、two's complement (补码)

“ 带星号 (*) 的条目超出本课程的要求范围, **仅做科普**。”

2. 课程讲义 Review

2.1. Undefined Behavior (未定义行为)

☒ Undefined Behavior (未定义行为) ->

UB 的出现意味着：**程序行为没有任何限制。**

常见的 UB 包括：

- **有符号整数溢出**
- 数组越界的内存访问（后续讲解）
- 空指针解引用（后续讲解）

2. 课程讲义 Review

2.1. Undefined Behavior (未定义行为)

☒ Undefined Behavior (未定义行为)

UB 的出现意味着：**程序行为没有任何限制。**

☒ Unspecified Behavior (未指定行为)

允许两种或以上的行为，且实现（编译器/平台）不要求记录每种行为的具体效果。
一个比较有代表性的例子是：函数参数中的求值顺序是一个 Unspecified Behavior

2. 课程讲义 Review

2.2. Integer Promotion (整型提升)

整型的 Rank 是什么？~~原来数据类型也有排名吗，硅基生物如此可怕...~~

在 C 语言中，Rank（等级） 是每个整数类型的一个**固有属性**，它决定了在整型提升 (Integer Promotion) 和寻常算术转换 (Usual Arithmetic Conversions) 中，不同类型之间如何进行隐式转换。

简单来说，Rank 定义了一个整数类型的“优先级”，Rank 高的类型在转换中“更强”，Rank 低的类型操作数会被转换为 Rank 高的类型。在涉及两个不同整数类型的操作中，编译器通过比较它们的 Rank 来决定将哪个操作数转换为何种类型，以**确保运算在一个“共同的真实类型 (common real type)”中进行**。

2. 课程讲义 Review

- 定义: Integer Promotion (整型提升) -> 是将 Rank 小于或等于 `int` 的任意整数类型, 隐式转换为 `int` 或 `unsigned int` 的过程。
- Example: 在 `==`, `+`, `-`, `*`, `/` 等二元算术运算符中, 若操作数类型 Rank 小于或等于 `int`, 则会进行整型提升。
- 右侧代码示例: 参与 `==` 运算的两个类型是 `char` 和 `unsigned char`, 它们的 Rank 都小于 `int`, 因此会被提升为 `int` 进行比较。实际上, `a == b` 在编译器中的实现是 `((int)a) == ((int)b)`。

```
#include <stdio.h>

int main(void){
    char a = 251;
    unsigned char b = a;

    if (a == b)
        printf("Same.\n");
    else
        printf("Not Same.\n");

    return 0;
}
```

2. 课程讲义 Review

2.2. Integer Promotion

整型提升的两个主要应用场景：

- **寻常算术转换 (Usual Arithmetic Conversions)**：在二元算术运算符中，若两个操作数类型不同，则会进行一系列隐式转换，以确保运算在一个“共同的真实类型 (common real type)”中进行。整型提升是寻常算术转换的一部分。
 - 还有浮点类型呢！
- **默认参数提升 (Default Argument Promotions)**：在可变参数函数（如 `printf`）中，传递给函数的实参会进行默认参数提升。整型类型会被提升为 `int` 或 `unsigned int`，而浮点类型会被提升为 `double`。
 - 例如 `printf("%d\n", (char)5);` 实际上传递给 `printf` 的是一个 `int` 类型的值 5。

是不是觉得很弯弯绕？实际上只需要记住一个核心需求：在一个**共同的真实类型中进行运算**。

2. 课程讲义 Review

2.3. Binary Number Representation (二进制数表示)

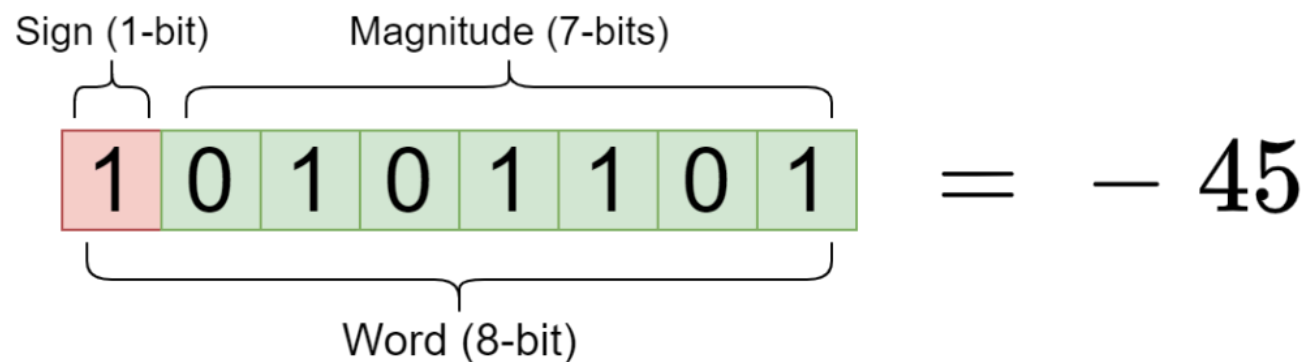
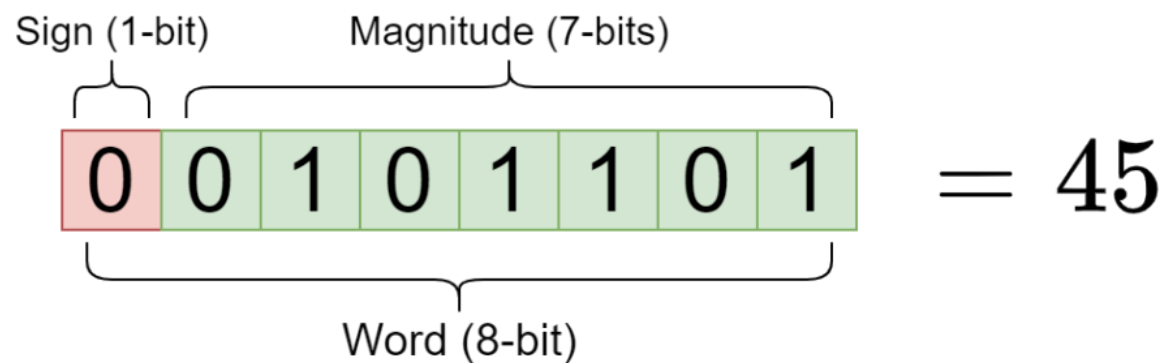
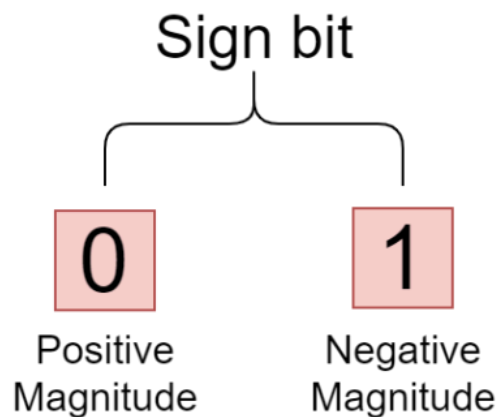
二进制数有三种常见的表示法：

- Sign-and-Magnitude (原码)
- One's complement (反码)
- Two's complement (补码)

“ 此内容在 SI100B EE Part 中亦有记载。

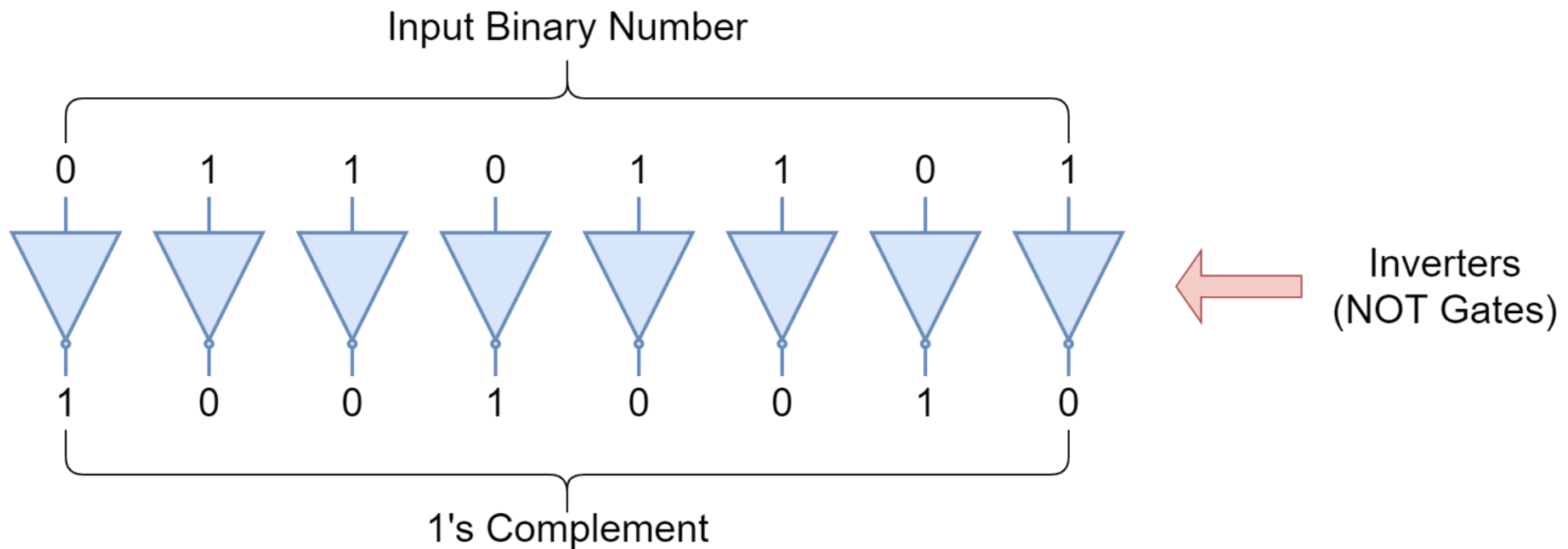
2. 课程讲义 Review

2.3.1. Sign-and-Magnitude (原码)



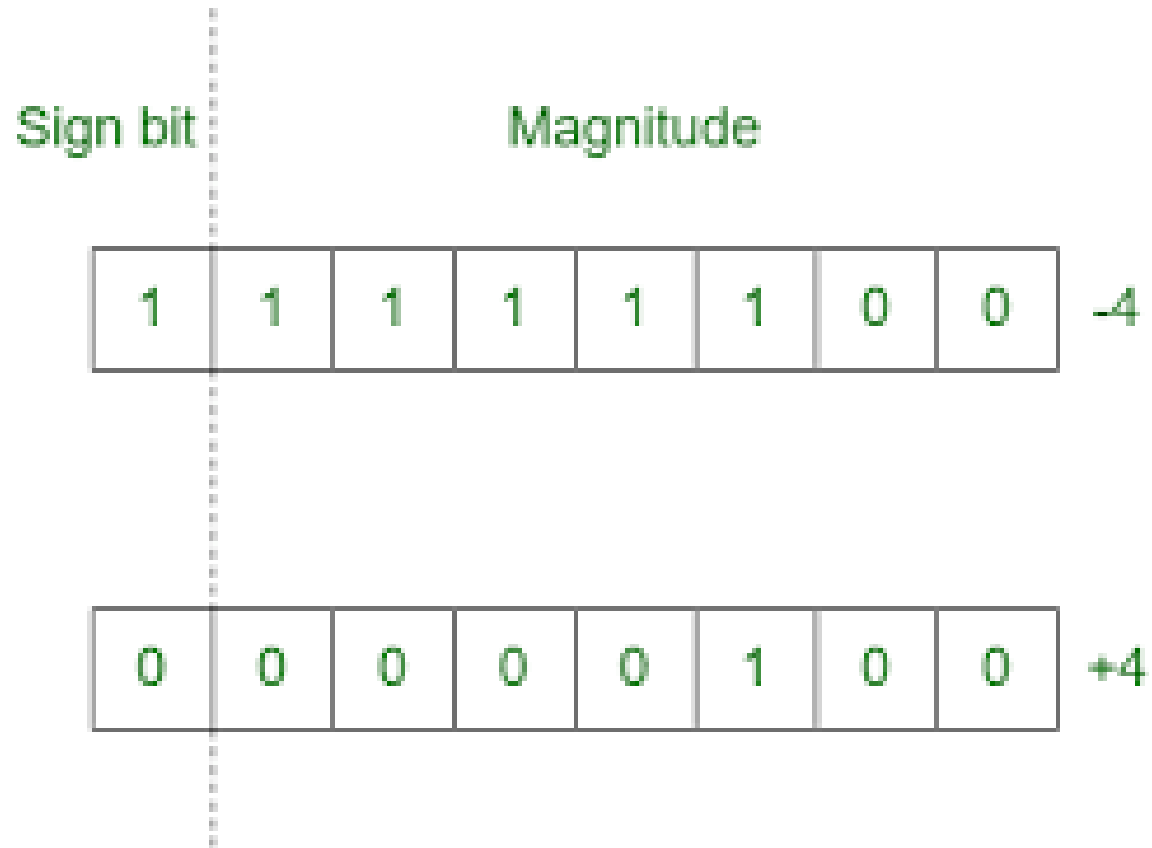
2. 课程讲义 Review

2.3.2. One's complement (反码)



2. 课程讲义 Review

2.3.3. Two's complement (补码)



3. Arithmetic Types

3. Arithmetic Types

Overview of This Section

这一部分的主要内容是 Arithmetic Types（算术类型）
->，即算术运算中使用的数据类型。

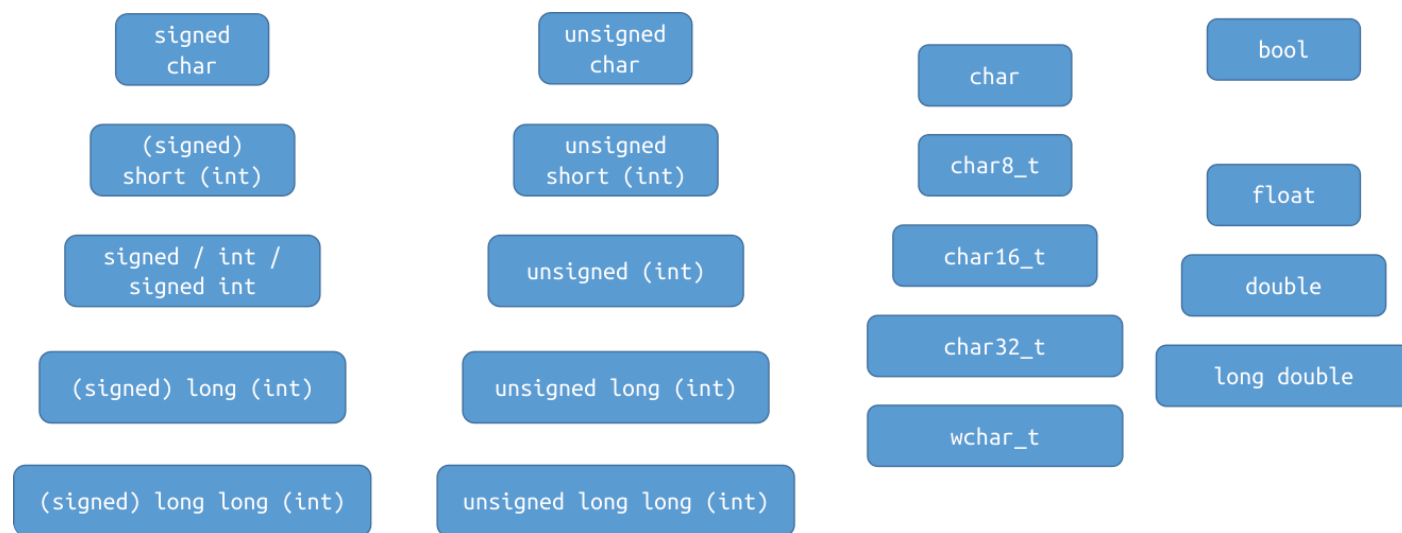


- 总览
- 整型宽度与 `sizeof`
- 浮点数误差

3. Arithmetic Types

3.1. Boolean, Character, Integer, Floating

“ 说明：在某些语境下，**字符（character）** 与 **布尔值（Boolean）** 也视作 **整数（integer）**。当然，事实上，这里的“某些”等于“几乎所有”。

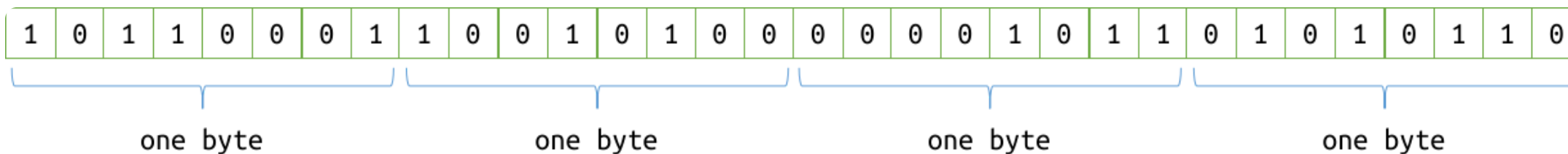


3. Arithmetic Types

3.2. 整型宽度与 `sizeof`

一个 32 位数示例:

$$2979269462_{10} = 10110001100101000000101101010110_2$$



3. Arithmetic Types

3.2. 整型宽度与 `sizeof`

- 大多数平台上 1 byte = 8 bits。
- `sizeof(T)` 返回类型 `T` 占用的字节数。
 - `sizeof()` 的返回类型是 `size_t` ->。(后续讲解)，其转换说明符为 `%zu`：

```
printf("%zu\n", sizeof(double));
```

- 若 MinGW 对 `%zu` 有问题，可转为 `int` 并使用 `%d`：

```
printf("%d\n", (int)sizeof(double));
```

3. Arithmetic Types

3.2. 整型宽度与 `sizeof`

类型	最小宽度	常见宽度
<code>short</code>	16 bits	16 bits
<code>int</code>	16 bits	32 bits
<code>long</code>	32 bits	32 或 64 bits
<code>long long</code>	64 bits	64 bits

- `sizeof(signed T) == sizeof(unsigned T)`
- **保证**: `sizeof(short) ≤ sizeof(int) ≤ sizeof(long) ≤ sizeof(long long)`。
- 你可以打印各类型宽度来观察不同机器/ABI 的实现差异。

3. Arithmetic Types

3.3. 浮点数误差 (Error of floating-points)

我们知道，有限位的小数无法准确表示无限不循环小数。

例如：只用十个数，表达 $1/7$ 只能是 0.1428571428 ，而非 $0.142857142857\dots$ 。当然，在七进制中， $1/7$ 可以被准确表示为 0.1_7 。

同理，浮点数作为一种**有限精度的二进制小数表示**，会无法准确表达大量实数。

例如经典的浮点误差梗 $0.1 + 0.2 \neq 0.3$ 。（实际上其结果是 0.30000000000000004 ）

“ 智械入侵的时候非常有用的知识 OvO



@qpomelo@octans.app
@realQPomelo

关注

???

我们怀疑您是人类，请完成机器人验证：



0.30000000000000004

☑ 您已通过验证，请继续

下一步

知乎 @尾鱼

3. Arithmetic Types

Exercise 1

定义函数 `midpoint`，接收两个浮点数并返回它们的平均值。注意选择**返回类型与参数类型**。

- 使用你的函数计算 `9007199254740993.0` ($2^{53} + 1$) 与 `9007199254740993.0` 的平均值。

“ 这还不简单？看我轻松拿下！



3. Arithmetic Types

Exercise 1

拿下!

```
double midpoint(double a, double b) {  
    return (a + b) / 2.0;  
}
```

补兑! 输出居然是 9007199254740992.000000?

这是怎么回事? ~~白毛小狗~~ 懵圈



3. Arithmetic Types

Exercise 1: Solution

- 标准的 `double` 使用 64 位浮点数格式：1 位符号 + 11 位指数 + 52 位尾数
- 实际精度为 53 位，即最多只能精确表达 2^{53} 这个整数
 - 更大的整数只能近似表达！
- $2^{53} + 1 = 9007199254740993$ 无法被 `double` 精确表示。
- 浮点数误差产生了！



3. Arithmetic Types

3.3. 浮点数误差 (Error of floating-points)

浮点数误差是 **不可避免 (inevitable)** 的，但我们可以通过一些数值技巧来减小浮点数误差带来的影响：

- **判断是否为零**：使用 $|x| < \epsilon$ (如 $\epsilon = 10^{-8}$)。

```
int isZero(double x, double eps = 1e-8) {  
    return (x < eps) && (x > eps);  
}
```

- **判断相等**：使用 $|a - b| < \epsilon$ 。

```
int isEqual(double a, double b) {  
    return isZero(a - b);  
}
```

4. Literals

4. Literals (字面值)

Overview of this Section

- 整数字面值 (Integer Literals)
- 浮点数字面值 (Floating-point Literals)
- 字符字面值 (Character Literals)
- ~~字符串字面值 (String Literals)~~
- ~~复合字面值 (Compound Literals)~~



4. Literals

Overview of this Section

- 每个 C 表达式在编译期都有确定类型。
 - 表达式是 operators 与 operands 的组合，描述一次计算。
- 例如： `int a = 42;` `double b = 3.14`
 - `a` 和 `b` 的类型分别是什么？
 - `42` 与 `3.14` 的类型又是什么？

4. Literals

Overview of this Section

像 `42` 这样直接写在代码中的值称为 **字面值 (literal)**。

- 字面值顾名思义，就是“把值写在脸上”的值（字面即是值）。
- 字面值是不可变的 (immutable)。

4. Literals

4.1. 整数字面值 (Integer literals)

Example: `42`, `100L`, `0LL`, `405ul`

- 后缀: `L` (long)、`LL` (long long)、`ul` (unsigned long) ——指示字面值的类型，大小写不敏感。
 - 建议不要混用小写 `l` 与大写 `L` (如 `ll` 或 `LL`)，统一使用 `L` 更清晰。
 - 后缀可以组合。
- 若无后缀，默认类型为 `int`。
 - 若超出 `int` 范围，会依次匹配 `int` → `long` → `long long` → `__int128`。若不支持 `__int128`，则这个字面值会成为一个 ill-formed (编译错误)。
- 注意: **并不存在“负字面值”**。例如 `-42` 是对字面值 `42` 应用一元负号运算所得。

4. Literals

4.1. 整数字面值 (Integer literals)

整数字面值的进制 (Base)

- 示例: `0xBAADF00D`、`052`、`0b101010` (C23)
- **前缀** (大小写不敏感):
 - 十六进制: `0x`
 - 八进制: `0`
 - 二进制: `0b` (自 C23 起)
 - (说明: C++ 自 C++14 起已支持二进制字面值)

4. Literals

4.2. 浮点数字面值 (Floating-point Literals)

- 示例: `3.14`、`3.14f`、`3.14l`、`1e8`、`3e-8`
- 后缀 (大小写不敏感):
 - 默认: `double`
 - `f`: `float`
 - `l`: `long double`
- 科学计数法总是浮点数。
 - 如 `1e8` 表示 10^8 , 但其类型是 `double` 而不是 `int`。

4. Literals

4.3. 字符字面值 (Character Literals)

Example: `'a'`、`'\n'`、`'1'`

- 在 C 中，类型为 `int`；在 C++ 中，类型为 `char`。

☒ 字符字面值

字符字面值是适于转换为字符类型的 `int`，或类型 `char8_t`（自 C23 起）、`char16_t`、`char32_t`、`wchar_t`（自 C11 起）的单个字符。

- 例：`char c = 'a';`
 - 变量 `c` 存的是整数 97，即字符 `'a'` 的 ASCII -> 码。
 - 字符与其 ASCII 码之间不存在“转换”——本质上它们就是同一数值。

4. Literals

Exercise 2

给定字符 `'c'` (且 `'c'` 为数字字符 `'1'`、`'2'`、`...`)，求其对应的整数值。

- 输入：单个数字字符 `c`。若 `c` 非数字字符，行为未定义。
 - 你可以输出一个错误！
- 输出：对应的整数值 (`int`)。



4. Literals

Exercise 2: Solution

```
int charToInt(char c) {  
    // 检查字符是否为数字字符  
    if (c >= '0' && c <= '9') {  
        return c - '0'; // 通过减去 '0' 的ASCII值得到对应的整数  
    } else {  
        // 如果输入不是数字字符，输出错误信息并返回一个特殊值（例如 -1）  
        printf("错误：输入字符 '%c' 不是数字字符! \n", c);  
        return -1; // 返回 -1 表示错误  
    }  
}
```



4. Literals

Exercise 3

如果你**只能**用 `getchar()` 读取**字符**，如何读取一个非负**整数**？

- `getchar()`：参见 CppReference ->
- 增强鲁棒性：若开头存在空白字符等噪声如何处理？
 - 使用标准库头文件 `<ctype.h>` 中的 character classification functions（字符分类函数） ->



5. 运算符 (Operators)

5. Operators

Overview of this Section

- Precedence (优先级)、Associativity (结合性)、Order of Evaluation (求值顺序)
- Bitwise operators (位运算符)



5. 运算符

5.1. Precedence（优先级）与 Associativity（结合性）

- **优先级**：决定**先**计算哪个运算符
 - 表达式 $f() + g() * h()$ 解释为 $f() + (g() * h())$ ，而非 $(f() + g()) * h()$ 。
- **结合性**：决定**相同优先级**的运算符如何分组
 - 表达式 $f() - g() + h()$ 解释为 $(f() - g()) + h()$ ，而非 $f() - (g() + h())$ 。
- 参考表： CppReference 运算符优先级/结合性 ->
- 经典问题： 见 Piazza ->

5. 运算符

5.1. Precedence（优先级）与 Associativity（结合性）

结合性：相同优先级的运算符如何分组

- **左结合**（Left Associativity）：从左向右分组
 - 例如 $f() - g() + h() \rightarrow (f() - g()) + h()$
- **右结合**（Right Associativity）：从右向左分组
 - 例如赋值运算符 $=$ ： $a = b = c$
 - 第一步： $a = (b = c)$
 - 第二步： $b = c$ 的结果是 b （且等于 c ）
 - 第三步： $a = b$

5. 运算符

5.2. Order of Evaluation (求值顺序)

- **非常重要**：求值顺序与优先级、结合性**相互独立**。
- **更重要**：除特别说明外，C 语言**大多数运算符的操作数求值顺序是未指定的 (unspecified)**。
 - 某些破学校的上古 C 语言课甚至会强行制定某些规则作为考试，看到这种教学请一律忽略！

5. 运算符

5.2. Order of Evaluation (求值顺序)

- 之前的例子 `f() - g() + h()`、`f() - (g() + h())` 中，`f()`、`g()`、`h()` 的调用顺序是什么？
 - 未指定 (unspecified)。
 - 很多时候你无法精确定义“未指定”的含义；理解编译器如何处理才是关键。详见 [order of evaluation - CppReference](#) ->。
- 另例：`func(f(), g(), h())`
 - 这里 `f()`、`g()`、`h()` 的调用顺序同样未指定。

5. 运算符

5.2. Order of Evaluation (求值顺序)

若两个表达式 `A` 与 `B` 的求值顺序未指定，且它们：

- 都修改同一变量的值，或
- 一个修改某变量值，另一个读取该变量值，

则构成**未定义行为 (undefined behavior)**。

- 例： `i = i++ + 2` 的结果是什么？
 - 一旦开始分析它是 `+1` 还是 `+2`，你就**掉入了未定义行为的陷阱**。
 - `++` 修改了 `i`，赋值也修改了 `i`，而两者的**先后顺序未指定**，因此是 **undefined behavior**。
 - 类比： `double pi = 3.14; printf("%d\n", pi);` (类型不匹配导致 UB)

5. 运算符

5.2. Order of Evaluation (求值顺序)

在常见运算符中，只有 4 个具有确定的求值顺序：&&、||、?:、,。

- && 与 ||：短路求值 (short-circuit / minimal evaluation / McCarthy evaluation)
 - 先计算左操作数；若结果已足以确定整体值，则右操作数不再求值。
- 条件运算符 `cond ? t : f`
 - 先计算 `cond`，再根据其真假计算 `t` 或 `f`；两者只会计算其一。
- 逗号运算符 ,
 - 依次计算多个表达式，整个逗号表达式的值为最后一个子表达式的值。

5. 运算符

Exercise 4

```
int main(void){  
    int a = 1;  
    int b = (a++, a + 100, a - 10);  
}
```

“ 上述程序中 `b` 的值是多少？（提示：逗号表达式取最后一个子表达式的值）



5. 运算符

Exercise 4: Solution

答案上 `b = -8!`

1. `a++`: 先使用 `a` 的值 (`1`), 然后 `a` 自增为 `2`
2. `a + 100`: 计算 `2 + 100 = 102` (但结果被丢弃, 因为不是最后一个表达式)
3. `a - 10`: 计算 `2 - 10 = -8`, 这是最后一个表达式, 因此作为整个逗号表达式的值, 赋值给 `b`

所以 `b` 的值是 `-8`。

5. 运算符

5.3. Bitwise Operators（位运算符）

~, &, ^, |, <<, >>, 以及复合赋值（compound assignment）&=, |=, ^=, <<=, >>=

运算符	名称	示例	含义
~	按位非（NOT）	~a	a 的逐位取反
&	按位与（AND）	a & b	a 与 b 的逐位与
、	、	按位或（OR）	a b a 与 b 的逐位或
^	按位异或（XOR）	a ^ b	a 与 b 的逐位异或
<<	左移（Left Shift）	a << i	a 的二进制位左移 i 位
>>	右移（Right Shift）	a >> i	a 的二进制位右移 i 位

5. 运算符

5.3. Bitwise Operators (位运算符)

- $\sim a$: 把 a 的每一位都翻转。
 - 例如 `10010110_two` 的按位非为 `01101001_two`。
- $a \& b$: 第 i 位只有在 a 与 b 的第 i 位**都为 1** 时结果为 1。
- $a | b$: 第 i 位只要 a 或 b 的第 i 位**至少一个为 1** 时结果为 1。
- $a \wedge b$: 第 i 位在 a 与 b 的第 i 位**不同时** 结果为 1。

例: $a \& 1$:

- 相当于 a 与 $\underbrace{0 \cdots 001}_{n-1 \text{ bits}}_{\text{two}}$ 做按位与,
- 等价于 $a \% 2$ (取奇偶)。

5. 运算符

5.3. Bitwise Operators (位运算符)

设 a 为无符号整数。

- $a \ll i$: 把 a 的二进制位**左移** i 位
 - $a \ll 1$ 等价于 $a * 2$; 一般 $a \ll i$ 等价于 $a * 2^i$
 - **左侧移出的位被丢弃**
- $a \gg i$: 把 a 的二进制位**右移** i 位
 - $a \gg 1$ 等价于 $a / 2$
 - **右侧移出的位被丢弃**

5. 运算符

Exercise 5: 如何得到整数 2^n ?

这时候我们并没有 `pow` 等等指数函数!

难道要做一个循环, 每次 `*2` 吗?



5. 运算符

Exercise 5: 如何得到 2^n ?

```
1 << n
```

- 但 1 是 `int` 字面值。
- 若 `n` 较大，你可能需要 `1u << n`、`1ll << n` 等（以避免溢出，并同时得到期望类型）。



5. 运算符

Exercise 6

如何获得无符号整数 x 的第 i 位？

- 设 $i \in [0, N)$, N 为 x 的类型的总比特数, 0 位为最右侧 (LSB)。

```
int test_bit(unsigned x, unsigned i) {  
    // TODO  
}
```

5. 运算符

Exercise 6: Solution

```
int test_bit(unsigned x, unsigned i) {  
    // 将 x 右移 i 位, 使目标位移动到最低位  
    // 然后与 1 进行按位与操作, 提取该位的值  
    return (x >> i) & 1;  
}
```

假设 $x = 13$ (二进制为 1101), 我们想获取第2位的值:

- $x \gg 2$ 得到 0011 (十进制为3)
- $(x \gg 2) \& 1$ 得到 $0011 \& 0001 = 0001$, 即 1。

5. 运算符

Exercise 7

如何翻转无符号整数 x 的第 i 位?

```
unsigned bit_flip(unsigned x, unsigned i) {  
    // TODO  
}
```



5. 运算符

Exercise 7: Solution

如何翻转无符号整数 x 的第 i 位?

```
unsigned bit_flip(unsigned x, unsigned i) {  
    // 创建一个只有第  $i$  位为 1 的掩码, 然后与  $x$  进行异或操作  
    return  $x \wedge (1U \ll i)$ ;  
}
```

- $1U \ll i$: 将无符号整数 1 左移 i 位, 创建一个只有第 i 位为 1 的掩码
- $x \wedge \text{mask}$: 通过异或操作翻转第 i 位
 - 如果第 i 位原来是 0, 则 $0 \wedge 1 = 1$ (翻转)
 - 如果第 i 位原来是 1, 则 $1 \wedge 1 = 0$ (翻转)



5. 运算符

Exercise 8

如何从无符号整数 `x` 中提取位区间 `[low, high)`?

```
unsigned bit_slice(unsigned x, unsigned low, unsigned high) {  
    // TODO  
}
```

“ 提示：考虑使用 Mask!



5. 运算符

Exercise 8: Solution

```
unsigned bit_slice(unsigned x, unsigned low, unsigned high) {  
    // 创建一个掩码，用于提取指定的位区间  
    unsigned mask = (1U << (high - low)) - 1;  
  
    // 右移 x 使目标位区间对齐到最低位，然后应用掩码  
    return (x >> low) & mask;  
}
```

Good luck and enjoy programming!



本次 Slides 中的所有示例代码都会被 Release 在 Piazza 上。表情包暂时不 Release。